Exp-1 BRESENHAM'S LINE DRAWING ALGORITHM.

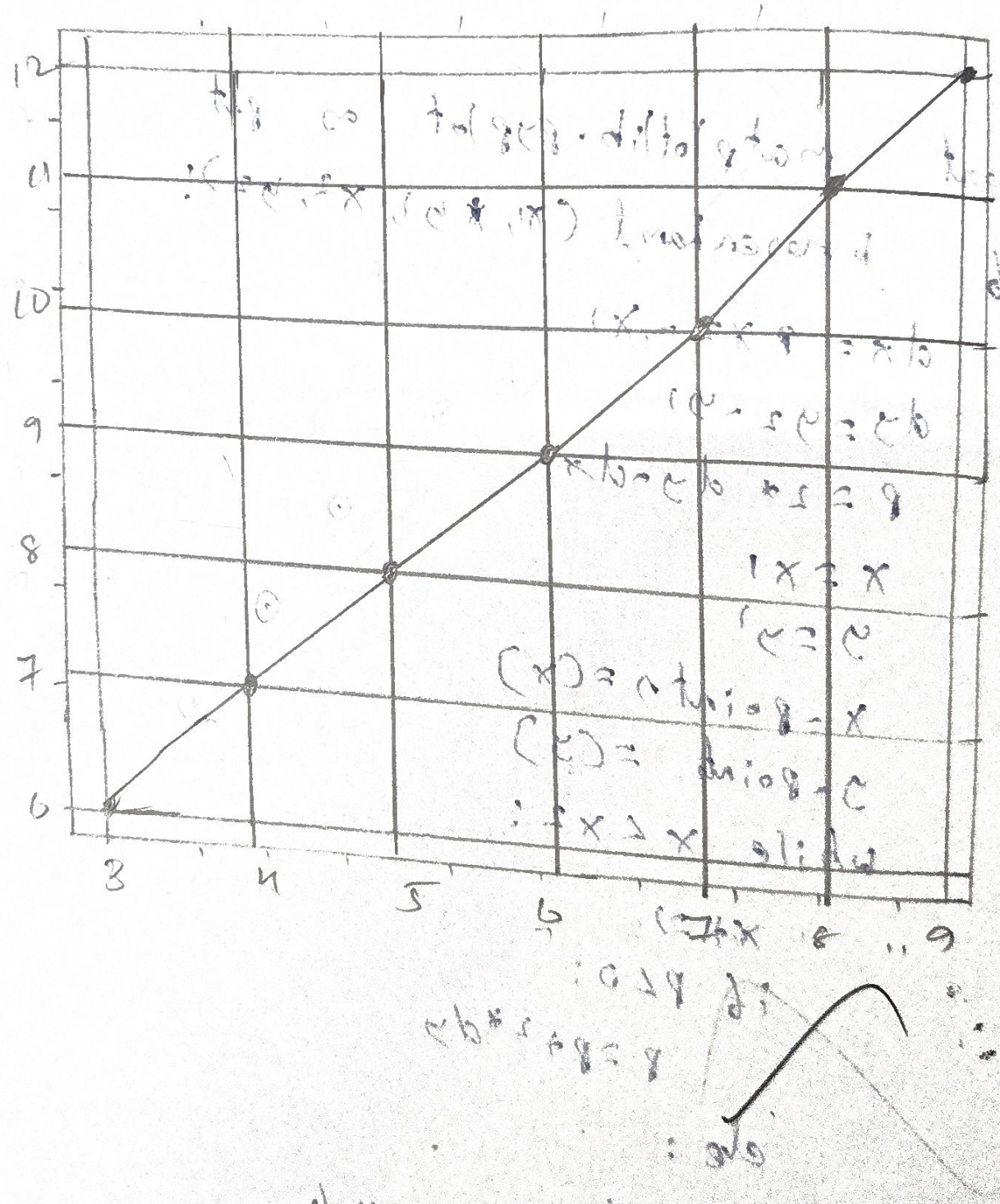
Aim:

To implement Bresenham's line drawing Algorithm (|m|<1) in Python for drawing a line when the slope.

Code:

```
import matplotlib.pyplot as plt
def bresenham(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    p = 2 * dy - dx
    x = x1
    y = y1
    x_points = [x]
    y_points = [y]
    while x < x2:
        x += 1
        if p < 0:
            p = p + 2 * dy
        else:
            y += 1
            p = p + 2 * dy - 2 * dx
        x_points.append(x)
```

Bresenham's Line Drawing Algorithm



```
y_points.append(y)
plt.plot(x_points, y_points, 'ro')
plt.title("Bresenham Line Drawing")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```

**Example.

x1, y1 = 2, 3

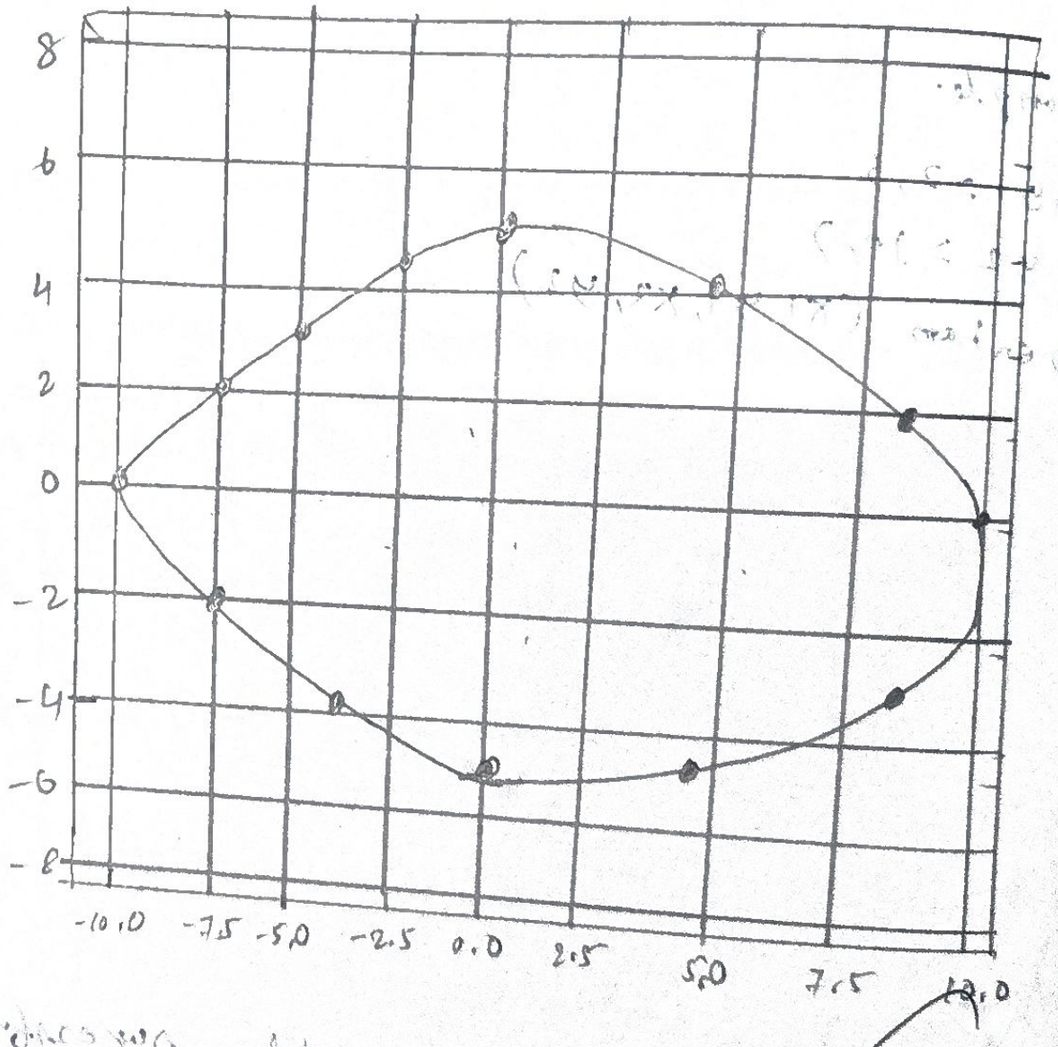
x2, y2 = 10, 7

bresenham(x1, y1, x2, y2)



Result:

The program was executed successfully. Bresenham's Line drawing Algorithm for the input where slope |m|<1.


8
6
4
2
0
-2
-4
-6
-8
-10.0
-7.5
-5.0
-2.5
0.0
2.5
5.0
7.5
10.0

Exp-2

BRESENHAM'S ~~LINE~~ ELLIPSE DRAWING ALGORITHM

Aim:

To implement Bresenham's ellipse drawing Algorithm in using Python Programming language.

code:

```
import matplotlib.pyplot as plt
import numpy as np
t = np.linspace(0, 2*np.pi, 100)
x = 10* np.cos(t)
y = 5* np.sin(t)
plt.plot(x, y, 'bo-')
plt.axis('equal')
plt.grid()
plt.show()
```

Result:

The Implementation of Bresenham's ellipse drawing algorithm was successfully executed and the output is verified.

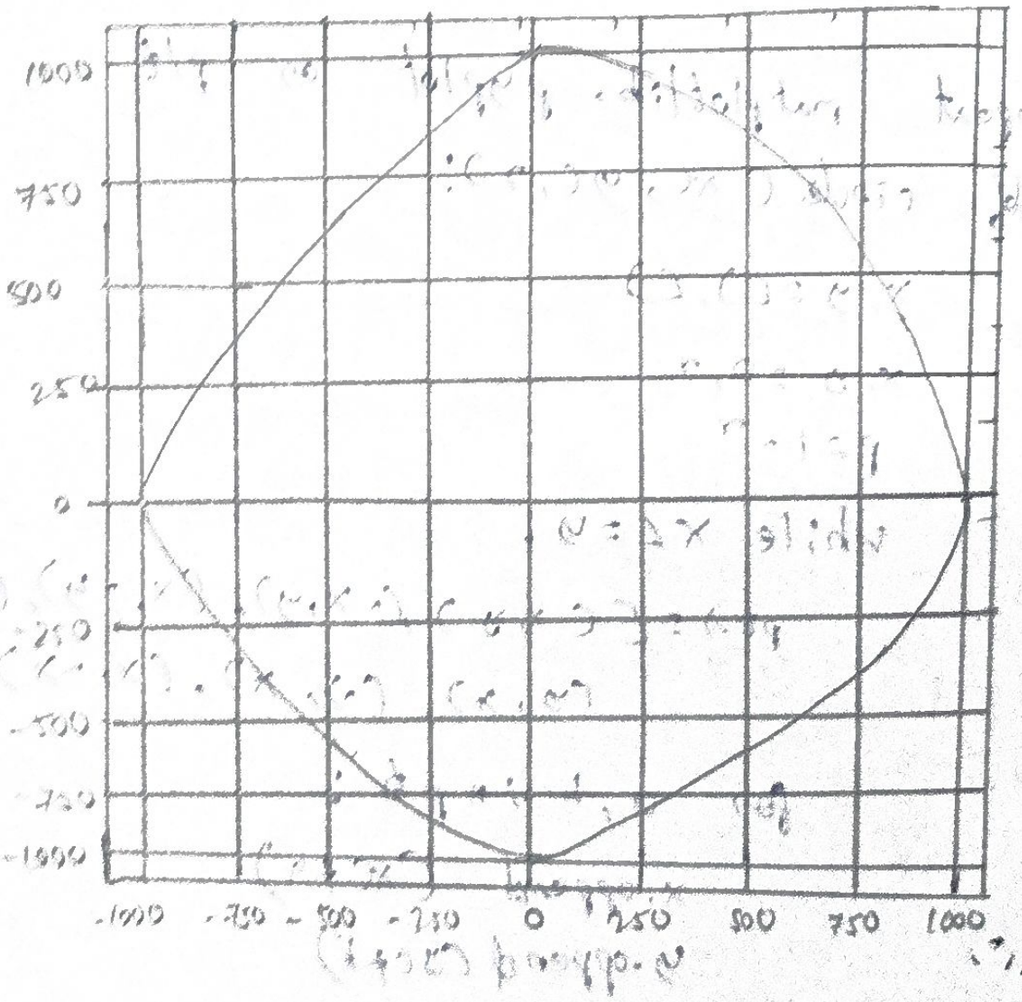
Exp-3 BRESENHAM'S CIRCLE DRAWING ALGORITHM.

Aim:

To implement Bresenham's circle drawing Algorithm using Python Programming language.

code:

```
import matplotlib.pyplot as plt
def circle(xc, yc, r):
    x, y = [], []
    x, y = 0, r
    p = 1 - r
    while x <= y:
        pts = [(x,y), (-x,y), (x,-y), (-x,-y),
               (y,x), (-y,x), (x,-x), (-y,-x)]
        for a,b in pts:
            x.append(xc+a)
            y.append(yc+b)
        x += 1
        if p < 0:
            p += 2*x+1
        else:
            y -= 1
            p += 2*(x-y)+1
```


1000
750
500
250
0
-250
-500
-750
-1000
-1000
-750
-500
-250
0
250
500
750
1000

```
return x, y

x, y = circle(0, 0, 10)
plt.plot(x, y, 'bo')
plt.axis('equal')
plt.grid()
plt.show()
```



Result:

Thus, the implementation of Bresenham's circle drawing algorithm is successfully executed and the output is verified.

Ex: No-4

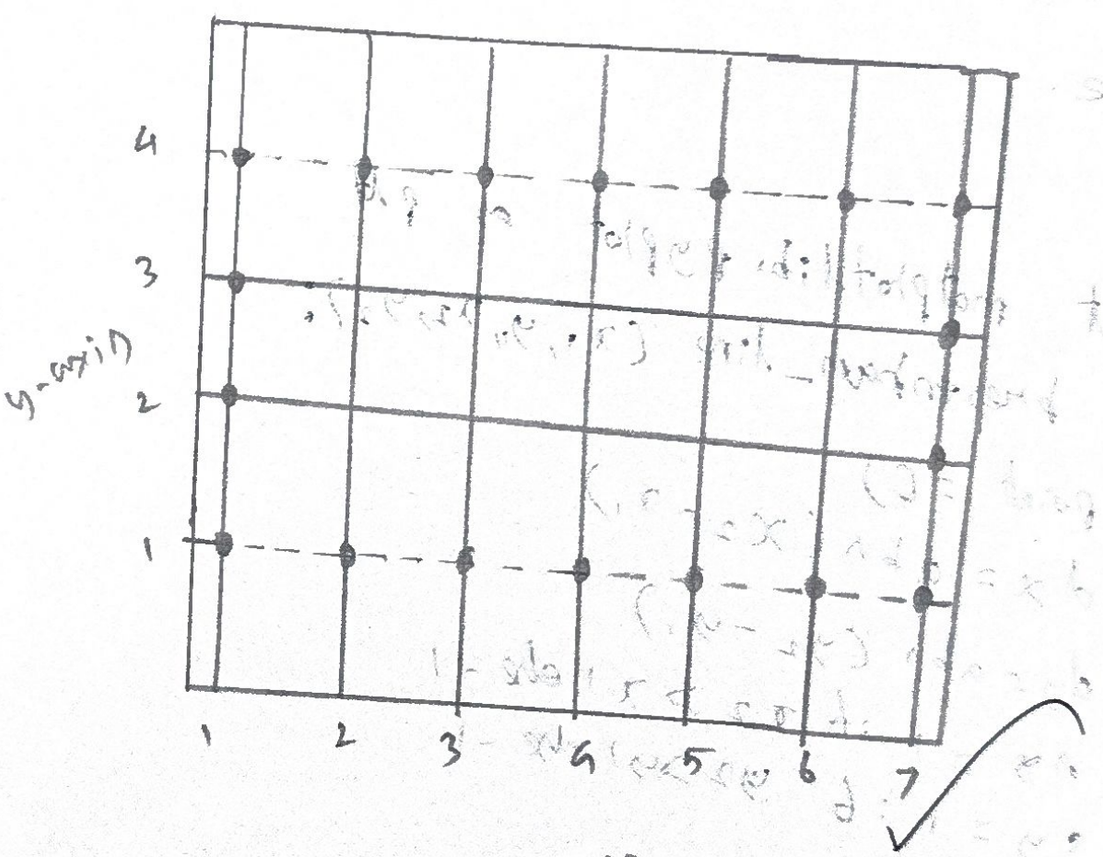
Bresenham's Rectangle Drawing Algorithm.

Aim:

To implement Bresenham's rectangle drawing algorithm using Python Programming language.

Code:

```
import matplotlib.pyplot as plt
def bresenham_line(x1, y1, x2, y2):
    point = []
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    sx = 1 if x2 > x1 else -1
    sy = 1 if y2 > y1 else -1
    err = dx - dy
    while True:
        points.append((x1, y1))
        if x1 == x2 and y1 == y2:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x1 += sx
```


4
3
2
1
y-axis
1
2
3
4
5
6
7
x-axis

```
    if e2 < dx:
        err += dx
        y1 += sy
return points

x = 1
y = 1
width = 5
height = 3

top_left = (x, y)
top_right = (x + width, y)
bottom_right = (x + width, y + height)
bottom_left = (x, y + height)

top = bresenham_line(*top_left, *top_right)
right = bresenham_line(*top_right, *bottom_right)
bottom = bresenham_line(*bottom_right, *bottom_left)
left = bresenham_line(*bottom_left, *top_left)

x_points = [p[0] for p in points]
y_points = [p[1] for p in points]

plt.figure(figsize=(6,5))
plt.plot(x_points, y_points, 'o-')
plt.title("Rectangle using Bresenham's
                Algorithm")
```

Ex-No:05

OLYMPIC LOGO USING MIDPOINT ALGORITHM

Aim:

To implement olympic logo using midpoint algorithm using python programming.

Program:

```
import matplotlib.pyplot as plt
def midpoint_circle(xc,yc,r):
    point = []
    x = 0
    y = r
    F = 1-r
    while x <= y:
        point += [(xc+x, yc+y), (xc-x, yc+y),
                  (xc+x, yc-y), (xc-x, yc-y),
                  (xc+y, yc+x), (xc-y, yc+x),]

        x += 1
        if p < 0:
            p += 2*x+1
        else:
            y -= 1
            p += 2*(x-y)+1
    return points
plt.figure(figsize=[8,6])
r=3
rings = [(3,6),(9,6),(15,6),(6,3),(12,3)]
for xc,yc in rings:
```

```
points = midpoint_circle (r1, r2, r)
x = [p[0] for p in points]
y = [p[1] for p in points]
plt.scatter (x, y, s=5)

plt.title ("Olympic logo using midpoint circle
            algorithm")

plt.axis ("Equal")
plt.grid (True)
plt.show()
```



Result:

Thus, to implementation of Olympic logo using midpoint algorithm is successfull.

Ex-No: 6

BUTTERFLY USING MIDPOINT ELLIPSE ALGORITHM.

Aim:

To implement the butterfly using mid point algorithm using python programming.

Program:

```
import matplotlib.pyplot as plt
def midpoint_ellipse(xc, yc, rx, ry):
    points = []
    y = ry
    x = 0
    rx2 = rx * rx
    ry2 = ry * ry
    p1 = ry2 - (rx2 * ry) + (0.25 * rx2)
    while (2 * ry2 * x) < (2 * rx2 * y):
        points += [(xc+x, yc+y), (xc-x, yc+y),
                   (xc+x, yc-y), (xc-x, yc-y)]
        x += 1
        if p1 < 0:
            p1 += 2 * ry2 * x + ry2
        else:
            y -= 1
            p1 += 2 * ry2 * x - 2 * rx2 * y + ry2
    return points
plt.figure(figsize=(8,8))
```

```
for args in [(5,9,5,4), (5,4,9,4),
             (13,9,9,4), (13,4,9,4),
             (10,7,2,7)]:
    points = midpoint_ellipse(args)
    x = [p[0] for p in points]
    y = [p[1] for p in points]
    plt.scatter(x,y,s=8)
plt.plot((9,7), (13,16))
plt.plot([11,13], [13,16])
plt.title("Butterfly using midpoint ellipse
           algorithm")
plt.axis("Equal")
plt.grid(True)
plt.show()
```



Result:

Thus, the implementation of butterfly using midpoint algorithm using python Programming is successfull.

Ex. No: 7

Implicit rate using midpoint algorithm.

Aim:

To implement midpoint circle using midpoint algorithm using Python programming

Program:

```
import matplotlib.pyplot as plt
def midpoint_circle(r,xc,yc):
    points = []
    x = 0
    y = r
    p = 1-r
    while x<=y:
        points += [(xc+x,yc+y),(xc-x,yc+y),
                   (xc+x,yc-y),(xc-x,yc-y),
                   (xc+y,yc+x),(xc-y,yc+x),
                   (xc+y,yc-x),(xc-y,yc-x)]
        x+=1
        if p<0:
            p+=2*x+1
        else:
            y-=1
            p+=2*(x-y)+1
    return points
plt.figure(figsize=(8,8))
```

```
face = midpoint_circle (10,10,8)
x = [p[0] for p in face]
y = [p[1] for p in face]
plt.scatter (x,y, s=8)
eg = midpoint_circle (7,13,1)
x = [p[0] for p in eg]
y = [p[1] for p in eg]
plt.scatter (x,y, s=20)
eg2 = midpoint_circle (13,13,1)
x = [p[0] for p in eg2]
y = [p[1] for p in eg2]
plt.scatter (x,y, s=20)
smile = []
for x in range (5,16):
    y = 8 - ((x-10)**2)//15
    smile.append ((x,y))
x = [p[0] for p in smile]
y = [p[1] for p in smile]
plt.scatter (x,y, s=15)
```

Result:

Thus, the implementation of smiley face using mid point algorithm is successful.

Ex-No:08

CONVERT HORIZONTAL ELLIPSE INTO VERTICAL

Aim:

To implement the conversion of horizontal ellipse into vertical ellipse using midpoint algorithm.

Program:

```
import matplotlib.pyplot as plt
def midpoint_ellipse (xc,yc,rx,ry):
    points =[]
    x=0
    y=ry
    rx2 =rx*rx
    ry2 =ry*ry
    p1=ry2-rx2*ry+0.25*rx2
    while 2*ry2*x<2*rx2*y:
        points +=[(xc+x,yc+y),(xc-x,yc+y),
                  (xc+x,yc-y),(xc-x,yc-y)]
        x+=1
        if p1<0:
            p1+=2*ry2*x+ry2
        else:
            y-=1
            p1+=2*ry2*x-2*rx2*y+ry2
    p2=(ry2*(x+0.5)**2+rx2*(y-1)**2-
        rx2*ry2)
```

```
while  y >= 0:
  point += [(x(+x, y(+y), (x(-x, y(+y),
            (x(+x, y(-y), (x(-x, y(-y)]

  y -=1
  if  p2>0:
      p2 += -2*rx2*y+rx2

  else :
      x+=1
      p2 += 2*ry2*x-2*rx2*y+rx2

  return points.

horizontal = midpoints - ellipse (rx,ry,6,3)
vertical = midpoint - ellipse (rx,ry,3,6)
plt.figure (figsize=(10,6))
x = [p[0] for p in vertical]
y = [p[1] for p in vertical]
plt.scatter (xy, y, s=10)
plt.title ("Horizontal ellipse to vertical
                Ellipse")
```

Result:

Thus, the implementation of the coversion of ellipse from horizontal to vertical using midpoint is succesful.

Exp No: 09

Scan Conversion of Polygon using Flood fill

Aim:

To implement Scan conversion of Polygon using flood fill algorithm.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
width = 500
height = 500
image = np.zeros((height, width, 3), dtype = np.uint8)*255
vertices = [(100,400),(400,100),(350,300)
            (250,400),(100,300)]
fig, ax = plt.subplots()
polygon = [v for v in vertices] + [vertices[0]]
polygon = [v[1] for v in vertices] + [vertices[0][1]]
ax.fill(polygon, polygon, color = 'lightblue')
ax.plot(polygon, polygon, color = 'black',
                          line width = 2)

ax.set_xlim(0,500)
ax.set_ylim(0,500)
ax.set_aspect('equal')
ax.set_title("Scan conversion of Polygon using
              flood fill")

plt.show()
```

Result:

Thus, the implementation of scan conversion of polygon using flood fill algorithm is successfull.

Ex-No:14

Aim:

To implement basic 2d transformations using 2D shapes using Python Programming language.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
triangle = np.array([[2,2],[5,2],[3,5],[2,2]])
tx, ty = 2,1
translation = triangle + [tx,ty]
sx,sy = 2,2
scaling = triangle * [sx,sy]
angle = np.radians(45)
rotation_matrix = np.array([[np.cos(angle), -np.
                  sin(angle)],[np.sin(angle),
                  np.cos(angle)]])
Rotation = triangle @ rotation_matrix.T
reflection = triangle * [1,-1]
fig,axes = plt.subplots(4,3,figsize=(16,7))
axes[0,0].plot(triangle[:,0],triangle[:,1],
axes[0,0].set_title("Original triangle")
```

```
axes[0,0].plot(translation[:,0], translation[:,1])
axes[0,0].set_title('Translation')
axes[0,1].plot(scaling[:,0], scaling[:,1])
axes[0,1].set_title('Scaling')
axes[1,0].plot(reflection[:,0], reflection[:,1])
axes[1,0].set_title('Reflection')
axes[1,1].plot(shearing[:,0], shearing[:,1])
axes[1,1].set_title('Shearing')

for ax in axes.flat:
    ax.axhline(0, color='black', linewidth=0.5)
    ax.axvline(0, color='black', linewidth=0.5)
    ax.grid(True)
    ax.set_aspect('equal')

plt.tight_layout()
plt.show()
```

Result:

Thus, the implementation of 2D transformation using 2D shapes using Python in numerical

Ex no:11

Polygon Clipping.

Aim:

To write a Program that clips a Polygon to a specified rectangular window using the Sutherland hodgman Polygon clipping algorithm and display the clipped polygon.

Program:

```
import matplotlip.pyplot as plt
LEFT, RIGHT, BOTTOM, TOP = 0, 1, 2, 3
def inside (p, edge, clip_win):
    x, y = p
    x min, xmax, ymin, ymax = clip_win
    if edge == LEFT:
        return x >= xmin.
    elif edge == RIGHT:
        return x <= xmax
    elif edge == BOTTOM:
        return y >= ymin.
    elif edge == TOP:
        return y <= ymax
    def intersect (p1, p2, edge, clip_win):
        xmin, xmax, ymin, ymax = clip_win
        x1, y1 = p1
        x2, y2 = p2
        if edge == it:
```

```
        x = xmin
        y = y1 + (y2-y1)*(xmin-x1)/(x2-x1)
    elif edge == RIGHT:
        x = xmax
        y = y1 + (y2-y1)*(xmax-x1)/(x2-x1)
    elif edge == BOTTOM:
        y = ymin
        x = x1 + (x2-x1)*(ymin-y1)/(y2-y1)
    elif edge == TOP:
        y = ymax
        x = x1 + (x2-x1)*(ymax-y1)/(y2-y1)
    return (x,y)
def clip_polygon(polygon, clip_win):
    output = polygon
    for edge in [LEFT, RIGHT, BOTTOM,
                 TOP]:
        input_list = output
        output = []
        if not input_list:
            if inside(p, edge, clip_win):
                if not inside(q, edge, clip_win):
                    output.append(intersect(q,p))
            output.append(p)
```

Result:

Thus, the implementation of polygon clipping is successfull and the output is verified.

Exp. No: 12

3D Transformation.

Aim:

To write a Python Program to Perform 3D transformation on the basic 3D objects.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection.

cube = np.array([ [0,0,0],[1,0,0],[1,1,0],[0,1,0],
        [0,0,1],[1,0,1],[1,1,1] [0,1,1]])

pyramid = np.array([[0,0,0],[2,0,0],[2,2,0],
                    [0,2,0],[1,1,2]])

cub_faces = [[0,1,2,3],[0,1,4],[1,2,4],[2,3,4],
             [3,0,4]]

def transformation(points, choice):
    if choice == 1:
        tx = float(input("Enter tx,ty,tz:"))
        ty = float(input("Enter ty:"))
        tz = float(input("Enter tz:"))
        matrix = np.array([tx, ty, tz])
        return points + matrix
```

```
elif choice == 2:
    x = float(input("Enter x: "))
    y = float(input("Enter y: "))
    z = float(input("Enter z: "))
    return points + np.array([x, y, z])

elif choice == 3:
    angle = np.radians(float(input("Enter angle
                                   in degrees: ")))
    rotation = np.array([[np.cos(angle), -np.sin(angle), 0],
                         [np.sin(angle), np.cos(angle), 0], [0, 0, 1]])
    return points @ rotation.T

return points
def display(points, faces, title):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
```

Result:

Thus, the implementation of 3D transformation of basic 3D objects is successful and the output is verified.

Ex-13:

3D colon and shading.

Aim:

To create and render 3D objects using polygon modelling and apply flat shading and basic colon.

Program:

```
import matplotlib.pyplot as plt
from mp_toolkit.mplot3d.art3d
import poly3D collection
import numpy as np
vertices = np.array [[0,0,0], [[0,0,1], [1,1,0],
           [0,1,0], [0,0,1], [1,0,1], [1,1,1], [0,1,1]]
faces = [[vertices [j] for j in [0,1,2,3]],
         [vertices [j] for j in [4,5,6,7]],
         [vertices [j] for j in [0,1,5,4]],
         [vertices [j] for j in [2,3,7,6]],
         [vertices [j] for j in [1,2,6,5]]]
```

Result:

Thus, the implementation of shading and rendering 3d objects within shading colon is successfull and output is verified.

Ex-14:

Multimedia Application.

Aim:

To develop a simple multimedia application that integrates images, audio, video into a unified user interface.

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import image, imagetk
from playsound import playsound
import cv2
import threading
def play_audio():
    play_sound "sample_audio.mp3")
def play_video():
    cap = cv2.videoCapture ("sample_video.mp4")
    while cap.isOpened():
        if not ret:
            break
        cv2.imshow ("video", frame)
        if cv2.waitKey(25):
            break
        cap.release()
        cv2.destroyAllwindows()
def load_images():
    img = image.open('sample_img.jpg')
```

Ex-No:15

Multimedia Application that Integrates Images, Sounds and video in simple UI

Aim:

To develop a simple multimedia application that integrates into a unified user interface using Python.

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import image, imageTk
import cv2
import threading
def play_audio():
    playsound('sample_audio.mp3')
def play_video():
    cap = cv2.videoCapture('sample_video.mp4')
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("video", frame)
        if cv2.waitKey(25) &
            break
        cap.release()
        cv2.destroyAllWindows()
```

```
def load_image():
    img = image.open('Sample_image')
    img = img.resize((300, 300))
    img_tk = imageTk.PhotoImage(img)
    panel.configure(image=img_tk)
    panel.image = img_tk
    btn_img.pack(pady=5)
    btn_audio.pack(pady=5)
    btn_video.pack(pady=5)
    window.mainloop()
```

Result:

A Multimedia application was successfully created.

Ex-No:16

Capture Video Audio from Webcam on microphone and Display on Multimedia Interface

Aim:

To develop a Program that capture like video from the webcam, Live audio from the microphone and display using multimedia interface

Program:

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk
def record_audio():
    duration=5
    fs=44100
    print("Recording Audio...")
    audio=sd.rec(int duration*fs), samplerate=fs. channels=2)
    sd.wait()
    write('recorded_audio, wave' fs, audio)
    print('Audio recording saved")

def capture_video():
    cap=cv2.videocapture(0)
    print("Press 'q' to stop video")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

Ex-17.

Create an application to send and receive multimedia messages (Text, Image, Audio)

Aim:

To develop a basic chat-like application sending and receiving text, image, and audio

Program:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading
def send_text():
    msg = text_entry.get()
    if msg:
        chat_box.insert(tk.END, "You: {msg}\n")
        text = entry.delete(0, tk.END)

def send_image():
    path = filedialog.askopenfilename(filetype =
            [("image files", "*jpg *png *jpeg")]

    if path:
        image_label.pack()
        chat_box.(tk.END)

def send_audio():
    path = filedialog.askopenfilename(filetypes =
            ("Audio files", "*mp3 *wav")

    if path:
        chat_box.insert
```

Result:

A multimedia messaging interface

was successfully created

Experiment 8

Aim: To model using Blender and apply basic shading and textures.

Aim: To create a simple 3D model by combining objects using Blender, and apply basic shading & texture.

Procedure:

1. Launch Blender
2. Model creation
   - Use add > mesh to insert basic shapes like cube, cylinder, plane etc..
   - Combine these objects to form a house or a similar structure
   - Use Edit mode for extruding and shaping
3. • Shading
   - Switch to shading workspace
   - Add a new material for each object
   - Adjust base color, specular and Roughness.
4. Texturing Mapping
   - In the Shader Editor, add an image Texture node
   - Load or extract image
   - Connect it to the base color
5. Lighting and camera.
   - Add light source
   - Position the camera to frame the model